

Markerless 3D Human Pose Tracking

Roland Bernard, 19598, rolbernard@unibz.it

Tutor Oswald Lanz

Domain Expert Marco Boschetti (Covision Lab)

Date September 2026

Abstract

The tracking of 3D human poses represents a crucial area in computer vision, with widespread applications across robotics, automated surveillance, healthcare, and entertainment. This project explores a tracking-by-detection framework designed to fuse multi-view 2D pose estimations from synchronized camera feeds into coherent, temporally consistent 3D skeletal trajectories. A system utilizing the YOLO26-pose model family for 2D keypoint extraction and extended Kalman filtering for 3D state estimation is developed, implemented, and evaluated. The project further explores the inclusion of skeletal constraints and a custom YOLO head to increase performance.

The resulting system was evaluated on eight video sequences from the CMU Panoptic dataset, benchmarking the impact of different system configurations on task performance and computational efficiency. Computational runtime analysis shows that the 2D detection phase dominates the processing, consuming between 67% and 93% of per-frame execution time and scaling linearly with the number of camera feeds. Furthermore, increasing the viewpoint density improves tracking and 3D reconstruction precision. For the smallest model configuration, scaling from 4 to 16 cameras reduces the Mean Per-Joint Position Error (MPJPE) from 8.25 cm down to 3.83 cm while also improving Multi-Object Tracking Accuracy (MOTA) from 0.78 to 0.93. The inclusion of skeletal constraints and tuning of the different components using the dataset also improves performance, with a 4 camera setup improving from a MOTA of 0.78 with the baseline to 0.90 for the fully tuned model. MPJPE also improves from 8.25 cm to 4.92 cm in the same setting.

1 Introduction

This capstone project focuses on markerless 3D human pose tracking. The estimation and tracking of 3D human poses is an important research area in computer vision, driven by its broad range of immediate applications across multiple domains. For example, in robotics and human-robot interactions it can find application in industrial environments and collaborative workspaces where robots must accurately perceive human poses and predict movements to maintain safe operational boundaries and avoid collisions with dangerous machinery (Svenstrup et al. 2009; Dondrup et al. 2015; Koppula et al. 2015). Similarly, for crowd control tracking of multi-person trajectories in public spaces facilitates anomalous behavior detection, crowd density planning, and automated surveillance (Sultani et al. 2018; Luo et al. 2021). Other impactful applications can be found in healthcare, such as gait analysis, automated fall detection, and remote

patient monitoring (Han et al. 2025; Gaya-Morey et al. 2024). Finally, human pose estimation and tracking can find applications also in the entertainment domain, such as the use of real-time motion capture for virtual avatars or natural body-based control interfaces for gaming (Mehta et al. 2020).

Despite the maturity of real-time 2D human pose estimation, transferring these methods to a robust and real-time capable 3D setting remains a challenging task due to issues such as depth ambiguities, severe self-occlusions, and the high perspective variance of human motion (Zheng et al. 2023; Nogueira et al. 2025). The primary objective of this project is to design, implement, and evaluate a computer vision architecture capable of tracking the skeletal movement of multiple people from 2D pose estimations extracted from multiple views by 2D human pose estimation models. The system should be able to fuse the information gained from the 2D video frames of multiple camera viewpoints into coherent and temporally consistent 3D pose trajectories.

For the evaluation of the result, this project will use a subset of the CMU Panoptic dataset (Joo, Liu, et al. 2015; Joo, Simon, et al. 2017), which contains videos captured from multiple calibrated cameras together with ground truth pose and tracking information for each frame. Using this evaluation set this project further aims to investigate the impact of model capacity as well as the viewpoint count on both the achieved tracking accuracy and the runtime efficiency of the system. For qualitative analysis the project also makes use of the SALSA dataset (Alameda-Pineda et al. 2015). This dataset does not contain ground truth pose annotations, but represents a more difficult application scenario.

The remainder of this report is structured as follows: Section 2 reviews the relevant theoretical background underlying the project. Section 3 gives an overview of the developed tracking system as well as detailed explanations of each component. Section 4 describes the used dataset and the evaluation strategy including evaluation metrics. Section 5 presents the quantitative and qualitative evaluation results. Finally, Section 6 concludes the report with a summary of the findings and outlines limitations, highlighting potential areas for further improvements.

2 Background

This section of the report covers relevant background knowledge necessary to understand the developed pose tracking system. A high level overview of a number of computer vision related topics is given.

2.1 Object Detection and Pose Estimation

In object detection the goal is to localize all objects, the number of which is not known a priori, in images (Lanz

Spring 2026). Like many computer vision recognition tasks, the field of object detection has evolved significantly with the introduction of deep learning (Lanz Spring 2026). Deep learning-based object detection has evolved through two primary paradigms. Anchor-based models, such as R-CNN or YOLO, rely on predefined bounding box priors of various scales and predicting offsets from these anchors. In contrast, anchor-free models predict bounding boxes directly from point features or keypoint combinations.

In two-stage detector models such as R-CNN, there are two separate inference stages (Bharati et al. 2019). The first stage generates a large number of candidate regions and the second stage then classifies these regions. Fast R-CNN (Girshick 2015) and Faster R-CNN (Ren et al. 2015) improve inference speed over the original R-CNN by using a shared feature backbone for both stages and by using a Region Proposal Network to create candidate regions.

In contrast to these two-stage detectors, the YOLO model used in this project is a single-stage detector (Jiang et al. 2022). It works by dividing the input image into multiple grids of different resolutions. Then a CNN based architecture is employed to extract features for each of the grid positions. Finally, one or more prediction heads are applied to the resulting feature maps to simultaneously predict both object presence and bounding box offsets from grid location defined anchors. The main advantage of single-stage approaches is that they are usually faster than two-stage approaches, although they often perform worse in practice (Lanz Spring 2026).

Once an object is detected, one can append extra heads to the feature maps to also predict other properties about the objects. An example of this approach is the Mask R-CNN architecture (K. He et al. 2017), which predicts instance masks for each detected object (Lanz Spring 2026). However, this approach can also be applied to Human Pose Estimation, where the additional output head of the network is instead used to predict keypoint locations (Maji et al. 2022). This approach is taken by the YOLO26 models used in this project (Jocher G. 2026; Sapkota, Cheppally, et al. 2025; Sapkota and Karkee 2025). Furthermore, as part of this work an additional custom head is trained for the YOLO26 model as explained in Section 3.7.1.

2.2 Object Tracking

Detecting objects or estimating poses in individual frames is insufficient for many applications (Lanz Spring 2026). Instead, what is often required is to associate detections with the physical objects they belong to and model how those evolve over time. This problem is called object tracking. The approach taken in this project is tracking-by-detection. This method can be divided into three essential steps, detection, association, and filtering. In detection, candidate objects are detected in individual frames. Then, these detections are assigned to concrete objects in the association step. Finally, filtering is used to determine the most probable state of the object given the associated detections. Filtering is responsible for refining the noise inherently present in the detection using probabilistic models and preceding observations. This subsection will focus on the association and filtering steps, while the detection step has already been discussed in Section 2.1.

In many applications multiple targets must be tracked

at the same time. To understand which detection belongs to which underlying object, association must be performed (Lanz Spring 2026). A typical way to proceed is to predict objects from the previous frame and to use those predictions to associate detections to the objects. This association can be performed using an association measure, which is a measure indicating the distance or similarity between a predicted object track and a detection. A naive approach, and also the one applied in this project, is to measure the overlap of the prediction and the detection (Bewley et al. 2016). Alternative approaches can include color histograms or optical flow and measure agreement (Lanz Spring 2026). It is also possible to use metric learning, where a deep learning model is used to extract features for computing the association measure.

After an association measure has been established, it is still necessary to actually perform the association (Lanz Spring 2026). A naive approach can be to consider all detections within some gating area of the prediction and then associate the closest of these detections to the prediction. The distance in this context can also consider the probabilistic model of the prediction to find most probable detections. However, this nearest neighbor approach can easily fail due to ambiguities. A better approach is to use a bipartite graph matching based framework. In this framework, the problem is modeled as a bipartite graph from predictions to detections, with edges weighted by the association metric. This can be formulated as the following integer linear programming problem:

$$\begin{aligned} \underset{\mathbf{Z}}{\operatorname{argmax}} \quad & \sum_{i=1}^M \sum_{j=1}^N s_{ij} z_{ij} \\ \text{s.t.} \quad & \forall j : \sum_{i=1}^M z_{ij} = 1 \quad \forall i : \sum_{j=1}^N z_{ij} = 1 \quad z_{ij} \in \{0,1\} \end{aligned}$$

where s_{ij} and z_{ij} are respectively the association metric and association between prediction i and detection j . This problem can then be solved either using a greedy algorithm or, as has been done for this project, using globally optimal algorithms such as the Hungarian method (Kuhn 1955). Finally, another possible approach to association is to use advanced graph-based deep learning networks.

After detections and objects have been associated, it is necessary to perform the filtering step (Lanz Spring 2026). It is crucial to understand that detections differ from the underlying state that we try to understand. Aside from being noisy, detections might also be missing some information of interest in the object. For example, object detections usually do not include velocity, and still we would like to infer it from the detections. When represented as using a Markov model, the state of a target at any given time step depends only on that target's state in the previous time step. Further, the observation or detection at any time step depends only on the targets current state. One way to evaluate this process is using recursive Bayesian estimation, also called Bayesian filtering.

Bayesian filtering is applied in two phases. First, the prediction step is used to model the target states distribution using only the past observations. Due to the conditional independence assumption of the hidden Markov model, this probability distribution of the hidden state can be computed

as followed:

$$\begin{aligned} p(\mathbf{x}_t | \mathbf{z}_{1:t-1}) &= \int p(\mathbf{x}_t, \mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}) d\mathbf{x}_{t-1} \\ &= \int p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{z}_{1:t-1}) p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}) d\mathbf{x}_{t-1} \\ &= \int p(\mathbf{x}_t | \mathbf{x}_{t-1}) p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}) d\mathbf{x}_{t-1} \end{aligned}$$

where $p(\mathbf{x}_t | \mathbf{x}_{t-1})$ defines our motion model, and $p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1})$ is the posterior distribution after applying all previous observations. After prediction, we want to incorporate the information gained by the new observation. This can be achieved as follows by applying Bayes' rule:

$$p(\mathbf{x}_t | \mathbf{z}_{1:t}) \propto p(\mathbf{z}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{z}_{1:t-1})$$

where $p(\mathbf{z}_t | \mathbf{x}_t)$ models the observation process.

The equations of the Bayesian filtering algorithm can be implemented by representing the probability distributions at each time step in parametric form, such as in Kalman filtering (Lanz Spring 2026; Bar-Shalom et al. 2001), or in nonparametric form, such as with particle filters. In a linear Kalman filter the motion and observation models are defined as:

$$\begin{aligned} \mathbf{x}_t &= \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_t) \\ \mathbf{z}_t &= \mathbf{H}_t \mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_t) \end{aligned}$$

The optimal closed-form solution is the standard Kalman Filter, which updates the mean $\bar{\mathbf{x}}_t$ and covariance Σ_t of $\mathbf{x}_t$ through linear projection and gain computation.

While the linear Kalman filter is applicable in cases where both the motion and observation model are linear, in our application of multi-view tracking, the measurement model is non-linear because 3D world coordinates must project onto a 2D camera plane. For this reason, this project makes use of extended Kalman filtering. We define the observation function to be $\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t$. The extended Kalman filter linearizes this non-linear function using a first-order Taylor expansion around the predicted state estimate $\bar{\mathbf{x}}_t$:

$$h(\mathbf{x}_t) \approx h(\bar{\mathbf{x}}_t) + \mathbf{J}_h(\mathbf{x}_t - \bar{\mathbf{x}}_t)$$

where $\mathbf{J}_h$ is the Jacobian matrix evaluated at $\bar{\mathbf{x}}_t$. In the update step the Jacobian matrix is used in place of the linear observation matrix $\mathbf{H}_t$ to update the covariance and calculate Kalman gain.

2.3 Projective Geometry

The dataset used in this project already contains camera calibration parameters for all camera views. Still, understanding the meaning of these parameters and how they relate to the image formation process is critical for the correct implementation for the triangulation algorithm as well as for modeling the observation process. The mapping of 3D points to 2D pixels is defined mathematically through the pinhole camera model (Lanz Spring 2026; Hartley et al. 2003). Unlike a physical pinhole camera, which would invert the image, the mathematical model places the image plane in front of the focal point for convenience. The full perspective projection model linking a homogenous 3D world coordinate

$\tilde{\mathbf{x}}_w$ to a homogenous 2D sensor pixel $\tilde{\mathbf{x}}_s$ is given by:

$$\tilde{\mathbf{x}}_s = \mathbf{K}[\mathbf{R}|\mathbf{t}] \tilde{\mathbf{x}}_w = \mathbf{P} \tilde{\mathbf{x}}_w$$

where $\mathbf{P}$ is the full 3×4 projection matrix. It decomposes into intrinsic parameters $\mathbf{K}$ and extrinsic parameters $\mathbf{R}$ and $\mathbf{t}$. For a perspective projection, 3D points $\tilde{\mathbf{x}}_s$ in the local camera coordinate system are mapped to the image plane by dividing them by their depth Z component.

The assumption of linear projection made by the above pinhole camera model, where straight lines remain straight, is violated by physical camera lenses, introducing distortions (Lanz Spring 2026). These must be modeled to accurately project points or undistort pixel coordinates. Two types of distortions are typically modeled. First, radial distortion causes points to bend outward or inward. It is modeled relative to the squared radius $r^2 = x^2 + y^2$. Second, tangential distortion arises when the lens is not perfectly parallel to the imaging sensor.

2.4 Multi-View Triangulation

Even once corresponding points across multiple camera views have been identified, their 3D position $\tilde{\mathbf{x}}_w$ must still be reconstructed from the 2D pixel locations. Rays back-projected from pixels will nearly never intersect perfectly in 3D space due to measurement noise. As such, a noise robust algorithm must be used that can recover 3D coordinates using multiple noisy 2D projections (Lanz Spring 2026; Hartley et al. 2003). If we denote $\tilde{\mathbf{x}}_i^s$ the homogeneous 2D observation in the i -th camera, and $\mathbf{P}_i$ the projection matrix of that camera, because $\tilde{\mathbf{x}}_i^s$ and the projected point $\mathbf{P}_i \tilde{\mathbf{x}}_w$ represent the same point up to scale, their cross product must be zero:

$$\tilde{\mathbf{x}}_i^s \times \mathbf{P}_i \tilde{\mathbf{x}}_w = 0$$

This cross product yields a set of three linear equations for each camera view. However, the third is a linear combination of the first two, so it can be discarded. By stacking the equations from $N \geq 2$ camera views, we obtain a homogeneous linear system $\mathbf{A} \tilde{\mathbf{x}}_w = 0$. Because $\tilde{\mathbf{x}}_w$ is defined only up to scale, we impose the constraint $\|\tilde{\mathbf{x}}_w\| = 1$. The optimal least-squares solution to this system is found by computing the Singular Value Decomposition of $\mathbf{A}$ and taking the right singular vector associated with the smallest singular value. Alternatively, it is also possible to transform the homogeneous system into an in-homogeneous system and solve it using standard least squares. This latter approach is taken in the project.

3 Methodology

This section of the report gives an overview of the developed system architecture and details the internal implementation of its different components.

3.1 Architecture Overview

Figure 1 shows the overall pipeline used in this project's tracking system. The core of the tracking pipeline runs a recursive estimation loop on incoming multi-view video

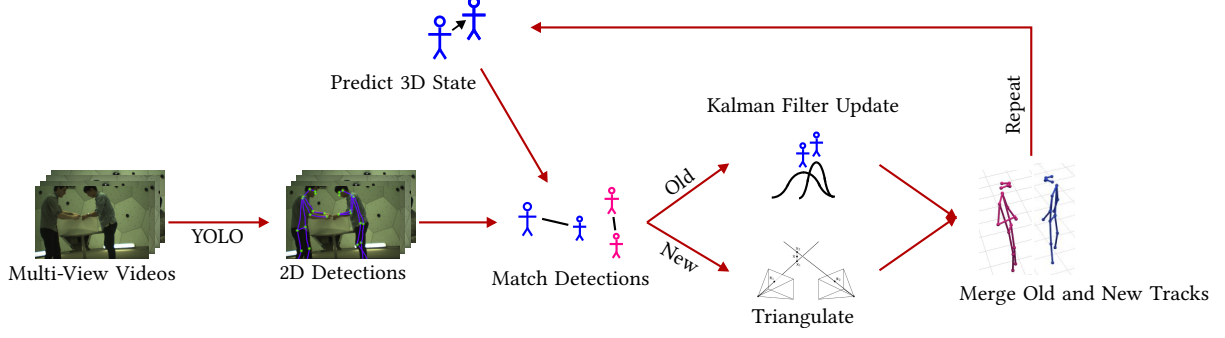


Figure 1: Architecture overview of the developed tracking system. Images are passed through a 2D pose estimation model to obtain the detections. These are matched against the predictions and the other cameras detections. New tracks are initialized using triangulation, and old tracks updated using Kalman filter update.

frames. The input to the system is a number of synchronized multi-view video feeds, and the final result is a 3D representation of the skeletal joint of each person in the scene.

First, multi-view videos are passed through the YOLO-based detection network, yielding 2D bounding boxes and keypoints. Concurrently, the tracking module predicts the current 3D state of all previously known tracks based on a continuous physics models. The keypoints of the predicted tracks are projected into the 2D camera planes and matched against the newly observed 2D detections. Detections matched to existing tracks trigger an extended Kalman filter update for the matched track to refine its 3D state. Novel 2D detections are matched between views, and triangulated to instantiate new 3D tracks if at least two views contain the new target. Finally, updated tracks and newly created tracks are merged, and the loop repeats for the next timestamp.

3.2 2D Detections and Uncertainty Estimation

During inference, images from each active camera view are batched and passed to the YOLO26-pose detector using PyTorch (Paszke et al. 2019; Ansel et al. 2024). The standard detector outputs 2D keypoint positions along with visibility confidence scores $c_k \in [0, 1]$. As a first filter we select only detections that have a confidence above τ_{obj} . Further, we filter out any detections for which fewer than N_{kpt} are visible with a confidence above τ_{kpt} . However, for performing the Kalman filter updates we need a probabilistic framework in which we model the prediction uncertainty as Gaussian noise. This need can not be satisfied by the confidence score alone. Section 3.7.1 describes how a custom YOLO head has been trained for directly outputting this uncertainty. However, for the baseline configuration of the system a more simple approach has been taken. To estimate the measurement noise covariance dynamically from the outputs of the YOLO model, the system relies on bounding box and visibility scaling. The variance σ_k^2 for each detected keypoint k is computed dynamically based on the bounding box magnitude and the joint visibility confidence. The specific formulation used is:

$$\sigma_k^2 = \sigma_{min}^2 + d_{bb}^2 \cdot \left(\frac{\sigma_{vis}^2}{\max(0, \tilde{c}_k) + \frac{\sigma_{vis}^2}{\sigma_{inv}^2}} \right)$$

where $\tilde{c}_k = \frac{c_k - \tau_{kpt}}{1.0 - \tau_{kpt}}$ is the confidence score normalized above the tuned keypoint threshold τ_{kpt} , d_{bb}^2 is the length of the diagonal of the bounding box, and $\sigma_{min}, \sigma_{vis}, \sigma_{inv}$ are tuning parameters specifying respectively the minimum variance, minimum bounding box dependent variance, and maximum bounding box dependent variance. This ensures that occluded joints are appropriately penalized with high uncertainty while visible joints get a lower variance. The keypoint detections are assumed to all be independent, so the final measurement noise covariance matrix is built as a diagonal matrix.

Further, before passing the resulting 2D keypoints to the rest of the tracking system, the 2D pixel coordinates are transformed into normalized camera coordinates utilizing the camera intrinsics and distortion coefficients as follows:

$$\mathbf{x}_{pixel} = \begin{bmatrix} u \\ v \end{bmatrix} \xrightarrow{\mathbf{K}^{-1}} \mathbf{x}_{distorted} \xrightarrow{\text{Iterative Undistortion}} \mathbf{x}_{normalized}$$

To maintain consistent uncertainty representation, we also transform the observation covariance matrix Σ_{pixel} for the 17 keypoints into normalized space using the following first-order approximation:

$$\Sigma_{norm} \approx \mathbf{K}^{-1} \Sigma_{pixel} \mathbf{K}^{-T}$$

The main motivation behind this approach is that it simplifies the triangulation for new tracks as well as the observation process modeling in the Kalman filter update for old tracks by removing the non-linear distortion effects. As an additional benefit, it becomes more efficient to compute not only the Kalman filter updates but also the computation of cost matrices for the association step, which need to perform projection from the internal 3D state. An alternative approach that achieves a similar trade-off is to remove distortion from images before they are passed to the detection model.

3.3 State Representation and Kalman Filtering

In the Kalman filter implementation of the project, the state for each person is tracked separately. The state vector is composed by modeling the human pose as a collection of 17 keypoints. The state vector $\mathbf{x} \in \mathbb{R}^{102}$ encapsulates the 3D coordinates and the 3D velocities for all 17 keypoints

resulting in a state vector shaped as:

$$\mathbf{x} = [\mathbf{p}_1^T, \dots, \mathbf{p}_{17}^T, \mathbf{v}_1^T, \dots, \mathbf{v}_{17}^T]^T$$

where $\mathbf{p}_k = [x_k, y_k, z_k]^T \in \mathbb{R}^3$ and $\mathbf{v}_k = [\dot{x}_k, \dot{y}_k, \dot{z}_k]^T \in \mathbb{R}^3$ are the 3D position and velocity vectors for keypoint k . Note that this is the state representation for the baseline configuration, while Section 3.6 explains how the state is modified for the introduction of skeletal constraints.

In the baseline system the state dynamics are modeled by a linear velocity process. Section 3.7.2 discusses how this is modified for the learned Kalman filter. In either case, the dynamics are modeled in continuous time. The decision was made to specify the dynamics in the continuous domain in order to be able to handle variable frame rates which could for example arise due to frames being dropped during computationally more demanding scenes. The continuous dynamics matrix $\mathbf{F}$ and process noise $\mathbf{Q}$ are mapped to discrete time steps Δt_i dynamically for each frame i using a second-order Taylor expansion based approximation:

$$\begin{aligned}\mathbf{F}_k &\approx \mathbf{I} + \mathbf{F}\Delta t_i + \frac{1}{2}\mathbf{F}^2\Delta t_i^2 \\ \mathbf{Q}_k &\approx \mathbf{Q}\Delta t_i + \frac{1}{2}(\mathbf{F}\mathbf{Q} + \mathbf{Q}\mathbf{F}^T)\Delta t_i^2\end{aligned}$$

The measurement function $h_c(\mathbf{x}) : \mathbb{R}^{102} \rightarrow \mathbb{R}^{34}$ acts as a non-linear pinhole projection, mapping the 102-dimensional 3D state back to a 34-dimensional observation of 2D coordinates in camera c :

$$h_c(\mathbf{x}) = [\pi_c(\mathbf{p}_1)^T \quad \pi_c(\mathbf{p}_2)^T \quad \dots \quad \pi_c(\mathbf{p}_{17})^T]^T$$

where $\pi_c(\mathbf{p}_k)$ is the perspective pinhole projection of the 3D point into normalized camera coordinates:

$$\pi_c(\mathbf{p}_k) = \frac{1}{Z_{c,k}} \begin{bmatrix} X_{c,k} \\ Y_{c,k} \end{bmatrix} = \frac{1}{\mathbf{R}_{c,3}\mathbf{p}_k + t_{c,3}} \begin{bmatrix} \mathbf{R}_{c,1}\mathbf{p}_k + t_{c,1} \\ \mathbf{R}_{c,2}\mathbf{p}_k + t_{c,2} \end{bmatrix}$$

Because multiple cameras observe the target simultaneously, the system concatenates these independent multi-camera views into a single centralized extended Kalman filter observation update instead of applying them sequentially. For a track observed by M cameras the observations are merged as follows:

$$\begin{aligned}\mathbf{z}_{merged} &= [\mathbf{z}_1^T : \mathbf{z}_M^T]^T \in \mathbb{R}^{34M} \\ \Sigma_{merged} &= \text{diag}(\Sigma_1, \dots, \Sigma_M) \in \mathbb{R}^{34M \times 34M}\end{aligned}$$

This formulation can handle targets visible by a subset of cameras naturally by simply omitting cameras without active detections. Note that, as is described in Section 3.6, the pseudo-observations implementing the skeletal constraints are also merged identically into the single update step.

Because the projection mapping and the skeletal constraint observations are both non-linear, the extended Kalman filter requires the computation of the Jacobian $\mathbf{H}_i \in \mathbb{R}^{34M \times 102}$ to linearize them. Rather than deriving and hard-coding this computation analytically, the system leverages the automatic differentiation capabilities of PyTorch (Paszke et al. 2019; Ansel et al. 2024) to compute the exact measurement Jacobian dynamically at runtime.

3.4 Data Association

Data association is performed in two separate steps. First, the detections for each view are matched against the existing 3D track predictions. This is performed independently for each view. Next, all detections that were not matched to any of the existing tracks must be matched between views in order to find opportunities for triangulation.

The association between detections and existing tracks is performed independently for each view by first projecting the prediction to the 2D camera plane. Note that we also project the 3D covariances by means of a first-order approximation using the Jacobian. A cost matrix is populated between projected predictions and YOLO derived detections. The distance metric is a modified Mahalanobis distance that adds the log-determinant of the combined covariance to discourage highly uncertain matches with low statistical power:

$$\begin{aligned}d_2(\mathbf{z}, \hat{\mathbf{z}}) &= (\mathbf{z} - \hat{\mathbf{z}})^T \Sigma^{-1} (\mathbf{z} - \hat{\mathbf{z}}) \\ C_{i,j} &= d_{\Sigma_{pred} + \Sigma_{obs}}(\mathbf{z}, \hat{\mathbf{z}}) + \log|\Sigma_{pred} + \Sigma_{obs}| - N_{dim}\theta_{mo}\end{aligned}$$

where $N_{dim} = 34$, and θ_{mo} is a threshold parameter. Note that we fill the cost matrix with zero padding to allow for unmatched tracks or detections. The optimal association is found by solving this cost matrix using the Hungarian algorithm (Kuhn 1955) via the SciPy library (Virtanen et al. 2020) implementation. This process can be interpreted as an implementation of maximum likelihood based model selection amongst all possible associations.

Detections that are not matched to existing tracks by the above method may belong to new people entering the scene. Since these detections lack 3D track priors, we must match them across the different camera views. To achieve this goal, this project employs an iterative greedy-optimal matching approach to associate 2D detections across views. The algorithm works by first building associations between the first two views, then iteratively adding views. During the iterative addition step we compute a cost matrix and solve for the optimal association using the Hungarian algorithm with the objective to minimize the reprojection error:

$$C_{cross} = \frac{1}{K} \sum_{k=1}^K (d_{\Sigma_k}(\mathbf{z}_k, \pi_k(\mathbf{X}_{3D})) + \ln|\Sigma_k|) - N_{dim}\theta_{mn}$$

where $\mathbf{X}_{3D}$ is the triangulated 3D point from the K associated views, and θ_{mn} is the new-track association threshold. Detections matched across at least two views are used to initialize new tracks via triangulation.

3.5 Track Creation and Deletion

To prevent spurious detections from creating false tracks, the lifecycle of each track is managed using a state machine. New tracks that have just been added via triangulation of detections start out as tentative tracks. If a tentative track is not matched to at least one detection in the next frame, it is deleted immediately. Once a tentative track has been observed for $\geq N_{age}$ consecutive frames, it is promoted to an active track. Active tracks are more robust to occasional missing detections. If an active track is not observed, its state is updated only using the Kalman filter prediction step and if enabled using skeletal constraints as described

in Section 3.6. However, if it remains unmatched for more than N_{inv} consecutive frames, it is deleted.

3.6 Skeletal Constraints

In the baseline configuration of the system as described above, there are no restrictions on the states that individual keypoints of a person can take and how they may evolve over time. However, it is possible to achieve better results by incorporating into the Kalman filter also prior knowledge about what state may be anatomically plausible and how they are allowed to change over time. Integration of such knowledge into the Kalman filter can be done in either the prediction or the update step. Integration into the prediction is possible by using a non-linear prediction function and linearizing it the same way as for non-linear observations. Alternatively, they can be encoded in the update step by making use of pseudo-observations. The inclusion of such constraints can improve not only the accuracy of predictions, but also improve the association step by ruling out matches that would violate the constraints.

This project integrates one such kind of knowledge, namely that the length of limbs for a single person remain constant. This is implemented by means of a pseudo-observation that is concatenated with the true observations as described in Section 3.3. To facilitate this mechanism, the state representation is augmented by adding the length of limbs:

$$\mathbf{x} = [\mathbf{p}_1^T, \dots, \mathbf{p}_{17}^T, \mathbf{v}_1^T, \dots, \mathbf{v}_{17}^T, l_1, \dots, l_{12}]^T$$

where each state value l_i represents the length of one or more limbs. These additional state elements interact with neither the positions nor the velocities and are given a low process noise covariance. Note that for this project a set of 12 lengths has been chosen by asserting symmetry between the left and right side of a person. This means that in addition to asserting a constant limb length, the resulting system also asserts symmetry.

The pseudo-observation that is added to the update step then consists of a small constant observation noise covariance, a constant zero vector for the observation mean, and the measurement function $h_{sc}(\mathbf{x}) : \mathbb{R}^{114} \rightarrow \mathbb{R}^{20}$. The measurement function extracts from the state the value of the constraint violation for each limb:

$$h_{sc}(\mathbf{x}) = [\rho_1(\mathbf{x}) \quad \rho_2(\mathbf{x}) \quad \dots \quad \rho_{20}(\mathbf{p})]^T$$

where $\rho_j(\mathbf{x})$ is the constraint violation for a single limb length:

$$\rho_j(\mathbf{x}) = \|\mathbf{p}_{a_j} - \mathbf{p}_{b_j}\|_2 - l_j$$

where a_j and b_j are the indices of start and end keypoints of limb j , while l_j is the length associated with limb j . Note that due to the symmetry assumption, there are more limb constraints than there are length values in the state vector.

By implementing the skeletal constraints as described above, the length values start out with large uncertainty. The pseudo-observations will induce covariance structures that link the length values to the keypoint locations and once the positions of the keypoints has been accurately determined,

also the length values will be known with high confidence. Since the process noise of these values is very low, and they do not otherwise interact with the other values in the prediction, their uncertainty will become and remain low.

3.7 Learning Parameters from Data

Up until now the system has been described in terms of algorithms that are configured using parameters, such as the dynamics and process noise matrices in the Kalman filter, the parameters for the observation noise computation in the YOLO-based detector, or the different thresholds used for association in the tracking system. While these values can be chosen from intuition and tuned manually, it would be desirable to be able to learn the appropriate parameters directly from data. This project makes use of the CMU Panoptic dataset as described in Section 4.1. Since this dataset contains both video from calibrated cameras as well ground truth pose annotation, it can be used for learning the parameters of the system and this was attempted in this project. A subset of 8 sequences has been selected for use in the evaluation of the resulting system. These test sequences were excluded from the training set, while the remaining scenes were used for training.

Ideally, the system would be trained end-to-end, such that all parts of the system would be able to adapt to the characteristics of the others. However, training the full tracking system end-to-end, including the YOLO models, would not only be computationally expensive, it also faces some additional issues. First, the tracking system is not fully differentiable. For example, the matching algorithm in the association steps are not differentiable and depend on non-differentiable thresholds. Further, while this non-differentiability may be accepted, training the system end-to-end would also lead to very unstable training, since the gradient flowing through multiple iterations of the Kalman filter prediction and update step, as well as through the observation function can easily vanish or explode.

Since end-to-end training is not easy to realize, this project instead opts to train components separately. Three different targets are identified for learning in this work. First, a custom YOLO head is trained that can predict directly not only the location of keypoints, but also a covariance matrix representing the confidence in the keypoint location predictions. Second, the parameters of the Kalman filter are learned. Finally, the thresholds for the association in the tracking system are tuned.

3.7.1 Custom YOLO Output Head

The YOLO26 family of models used for 2D pose estimation in this project perform well at detecting persons and predicting the location of visible keypoints. However, for the integration in the Kalman filter based tracking system there are two major drawbacks of the standard pretrained model. First, the model does not natively output its confidence in the keypoint location predictions. It only outputs a visibility score that represents whether the keypoint is visible or not. While one may naively assume that this score could be transformed into an uncertainty score, this is not without drawbacks since not all visible or invisible keypoints have the same uncertainty. The second limitation of the standard

YOLO models is that they do not even attempt to guess at the location of invisible keypoints. Instead, the keypoint locations returned by the model for invisible keypoints are largely not meaningful. However, in many situations it would be possible to make some educated guess about the location of the keypoints, even if with large uncertainty, which would lead to improved tracking performance.

To resolve these limitations, a custom YOLO head has been implemented and trained for this project. The head is applied to the same features at the output of the YOLO model that are also used for the computation of the standard YOLO pose head. The custom head outputs five values for each keypoints. The first two outputs for each keypoint encode the x and y offset from keypoint location predicted by the standard pose head. The next three components form a 2×2 lower triangular matrix L such that LL^T represents the covariance matrix encoding the uncertainty of that keypoints position prediction. The model is trained using a linear combination of mean-squared-error and negative-log-likelihood loss $L = L_{\text{NLL}} + \lambda L_{\text{MSE}}$, where λ has been set to 0.1 for the training performed in the project. The loss is computed by first performing a bipartite matching between the annotation keypoints and the standard YOLO keypoints to find the correct associations.

To perform the training a dataset with human pose annotations is required. Since we want the model to predict not only visible, but also invisible keypoints, even those that may potentially be outside the image boundaries, the COCO dataset used for the standard YOLO training can not be used. Instead, a custom dataset has been extracted from the CMU Panoptic data. For each training sequence, every second a single frame has been extracted at random from one of the cameras. These frames have then been annotated by projecting the ground truth 3D pose information onto the image plane of the selected camera view.

3.7.2 Learning the Kalman Filter

The parameters of the Kalman filter can also be learned from data. For this project we want to learn both the continuous dynamics matrix F and process noise Q . Further, also the initial state vector, the state covariance matrix, and the constraint observation matrix and constraint variances are learned. The training is performed by sampling from ground truth trajectories of a single person. From these ground truth sequences we then generated synthetic observations, which are generated using a known observation noise variance. After having generated the noisy observation sequence, the Kalman filter is run on those observations and the predicted positions of each keypoint recorded for each frame. The recorded predictions for each frame are then compared with the ground truth annotations, and the mean-squared-error is used as a loss function. Since training in this way can quickly become unstable since the Kalman filter contains many highly non-linear operations, the training has been initialized using the manually defined linear-velocity physics derived filter parameters.

The dataset for the training of the Kalman parameters, like that for the custom YOLO head, has been extracted from the CMU Panoptic dataset. For each sequence in the training set all track annotations have been extracted and separated by person. The tracks have further been split whenever

some keypoints are marked as unknown in the annotations. The tracks are then split into samples of 100-frame-long non-overlapping sequences. These resulting sequences serve as the dataset for the Kalman filter learning.

3.7.3 Tuning Other Parameters

There are some remaining parameters that we may also want to tune from the data. Specifically, in this project the τ_{obj} , θ_{mo} , and θ_{mn} threshold parameters have been tuned using a grid search. For each considered value in the grid search the tracking system was applied to a small sample of training sequences. MOTA and MPJPE were computed for each combination of values and the one reaching the best scores was selected as the tuned parameters.

4 Experimental Evaluation

This section of the report will focus on how the proposed system has been experimentally evaluated. Further it covers the dataset used for the experiments and the selected evaluation metrics.

4.1 Dataset Description

For the development and evaluation of the tracking system in this project, the CMU Panoptic Dataset (Joo, Liu, et al. 2015; Joo, Simon, et al. 2017; Simon et al. 2017) has been used. This is a large-scale dataset that was originally recorded to facilitate research in multi-view social motion capture. Figure 2 shows a sample of some still images extracted from the video sequences contained in this dataset.

The complete dataset includes 84 video sequences totaling approximately 11.5 hours of video captured in a specialized 5.49m diameter dome-shaped room equipped with inward-facing cameras attached to the outside panels. In total, the recording hardware consists of 511 calibrated cameras, 480 VGA cameras recording at a 640×480 resolution at 25 fps and 31 HD cameras recording at a 1920×1080 resolution at 30 fps. The room also contains 10 Kinect II (Kurillo et al. 2022) sensors and 5 DLP projectors. Because of the geometry of the room and the placement of cameras, no single camera is able to view the complete space. This means that while each point in the space is visible from multiple cameras, each camera only captures a partial view of the environment, introducing occlusion and field-of-view limits related challenges for the tracking task.

Crucially, the dataset also includes ground truth 3D skeletons and tracks that were generated by the CMU researchers using algorithms that leverage the entirety of the camera array (Joo, Liu, et al. 2015). Ground truth annotations of the dataset are provided in the COCO19 format. To align them with the outputs of the YOLO-based pose estimation networks used in this project (Jocher G. 2026), the ground truth data was pre-processed by converting it into the COCO17 format (T.-Y. Lin et al. 2014). This was achieved by discarding the two extra joint positions. The dataset also includes the calibration parameters for all available camera views, including camera extrinsics, intrinsics, and distortion parameters.

Due to computational and time constraints, running the tracking system across all the sequences for the dataset

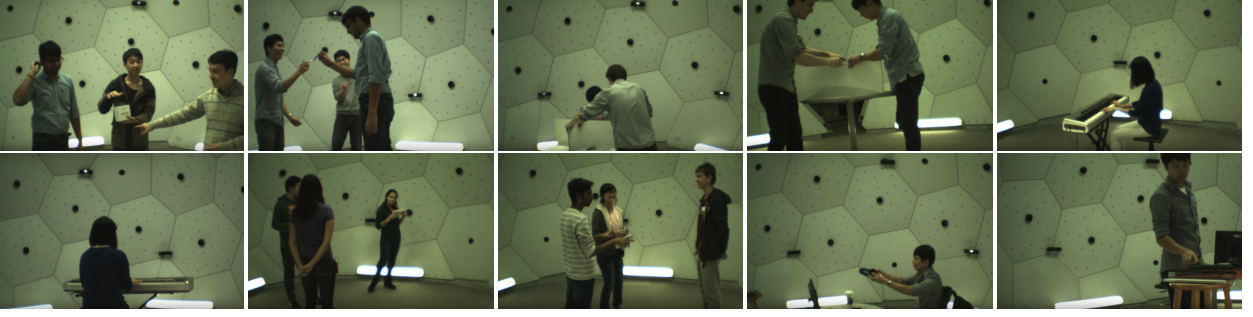


Figure 2: A series of example images extracted from the video sequences of the CMU Panoptic dataset.

was not reasonable. For this reason, a subset of the dataset was selected for use as the testing set. These sequences were also excluded from the training of the system components as explained in Section 3.7. The following eight sequences were reserved for testing: 160224_haggling1, 160906_ian1, 160906_pizza1, 161029_build1, 161029_sports1, 170221_haggling_m3, 170915_office1, and 171026_pose3. These sequences have been selected to cover a wide variety of different and challenging scenarios. Furthermore, evaluation was performed using only a subset of all camera views. A maximum of 16 VGA-type cameras were utilized per video sequence. These 16 cameras were selected based on the official dataset download script¹, which ensures they are approximately uniformly distributed around the dome to provide maximum spatial coverage with a minimal camera count.

In addition to the CMU Panoptic dataset, for some qualitative analysis and demos also the SALSA dataset (Alameda-Pineda et al. 2015) has been used. This dataset represents a much more difficult scenario for the system, however, because it does not contain any ground truth pose annotations, it could not be used for the quantitative evaluations.

4.2 Evaluation Procedure

For this project the system has been evaluated by comparing its 3D tracking predictions frame-by-frame against the ground truth annotations in the CMU Panoptic dataset. The experiments were run across all eight selected sequences mentioned in Section 4.1, evaluating various combinations of YOLO pose estimation models, testing YOLO26n-pose, YOLO26s-pose, YOLO26m-pose, and YOLO26l-pose, spacial view densities of 4, 8, 12, and 16, as well as testing different configurations of the system, such as with or without skeletal constraints. Note only the YOLO26n-pose model has been tested with more than 4 cameras and that the more advanced configurations of the system have also only been tested in the 4 camera setting. All evaluations were performed on AMD Ryzen AI 9 HX 370 CPU with most PyTorch (Paszke et al. 2019; Ansel et al. 2024) operations running on the integrated AMD Radeon 890M GPU.

For the baseline system configuration, parameters of the system were initialized heuristically rather than via exhaustive grid-search or end-to-end learning. While these rigid parameter values are functional, the system performs

better when using the components that were learned from data as shown in Section 5.

For the baseline system, detection thresholds for the YOLO based detector were set to $\tau_{obj} = 0.5$ and $\tau_{kpt} = 0.25$, using also $N_{kpt} = 3$ as the minimum number of visible keypoints. The minimum pixel variance has been set to $\sigma_{min}^2 = 16.0$ pixels², with visibility and invisibility scale variances set to $\sigma_{vis}^2 = 0.005$ and $\sigma_{inv}^2 = 5$. The matching thresholds for both cross-view and prediction-view matching have both been set to $\theta_{mn} = \theta_{mo} = 0$. For the tracking lifecycle we configured the age for tentative track promotion to $N_{age} = 4$ and the maximum duration without detections for active tracks to $N_{inv} = 10$. Finally, the continuous physics matrices have been set as follows:

$$\mathbf{F} = \begin{bmatrix} \mathbf{0}_{51 \times 51} & \mathbf{I}_{51 \times 51} \\ \mathbf{0}_{51 \times 51} & -0.2 \cdot \mathbf{I}_{51 \times 51} \end{bmatrix}, \mathbf{Q} = \begin{bmatrix} \sigma_p^2 \cdot \mathbf{I}_{51 \times 51} & \mathbf{0}_{51 \times 51} \\ \mathbf{0}_{51 \times 51} & \sigma_v^2 \cdot \mathbf{I}_{51 \times 51} \end{bmatrix}$$

with $\sigma_v^2 = (3\frac{m}{s})^2$ and $\sigma_p^2 = (1cm)^2$. This matrix models constant velocity dynamics with a damping term of -0.2 on velocity to stabilize motion predictions.

4.3 Evaluation Metrics

To measure tracking and reconstruction performance, this project makes use of standard metrics from the Multi-Object Tracking and 3D Pose Estimation literature (Bernardin et al. 2008; Ionescu et al. 2013). All measures are reported in centimeters, and we use a mean joint error of 15 centimeters as the threshold for matches for all metrics except MPJPE.

Mean Per-Joint Position Error (MPJPE) MPJPE measures the average 3D Euclidean distance between predicted and ground truth keypoints across all matched frames and joints.

$$\text{MPJPE} = \frac{1}{N_{matched}} \sum_{i=1}^{N_{matched}} \left(\frac{1}{17} \sum_{k=1}^{17} \|\mathbf{p}_{k,i}^{pred} - \mathbf{p}_{k,i}^{gt}\|_2 \right)$$

For this metric we do not apply a matching threshold, instead matching all predictions against the best matching ground truth track via Hungarian method based assignment.

Multi-Object Tracking Accuracy (MOTA) MOTA evaluates overall tracking performance by accounting for three types of errors, false positives, false negatives, and

¹<https://github.com/CMU-Perceptual-Computing-Lab/panoptic-toolbox/blob/master/scripts/getData.sh>

Model	# Cameras	Detection	Matching	EKF Update	EKF Prediction	Total
YOLO26n-pose	4	0.0411	0.0131	0.0069	0.0005	0.0616
	8	0.0850	0.0236	0.0102	0.0007	0.1195
	12	0.1403	0.0280	0.0118	0.0008	0.1809
	16	0.1922	0.0355	0.0163	0.0008	0.2448
YOLO26s-pose	4	0.1137	0.0139	0.0083	0.0007	0.1365
YOLO26m-pose	4	0.2866	0.0140	0.0087	0.0007	0.3100
YOLO26l-pose	4	0.3408	0.0143	0.0090	0.0007	0.3648

Table 1: Average per-frame runtime of the tracking system for different camera and model configurations. Times are given in seconds.

Model	# Cameras	MOTA	MOTP	MPJPE	Precision	Recall
YOLO26n-pose	4	0.781	5.488	8.250	0.890	0.890
	8	0.924	3.235	3.341	0.939	0.994
	12	0.926	3.066	3.146	0.939	0.996
	16	0.920	2.749	2.834	0.934	0.996
YOLO26s-pose	4	0.785	5.151	8.616	0.888	0.899
YOLO26m-pose	4	0.807	4.976	7.466	0.897	0.913
YOLO26l-pose	4	0.809	4.792	7.674	0.898	0.915

Table 2: Evaluation results for the baseline configuration with different camera and model configurations. MOTA, MOTP, Precision, and Recall have been computed with an average of 15 centimeters Euclidean distance as the matching threshold. MOTP and MPJPE are given as mean per-joint Euclidean distance in centimeters. The best result in each column is highlighted in bold.

identity switches:

$$\text{MOTA} = 1.0 - \frac{\sum_t \text{FN}_t + \text{FP}_t + \text{IDS}_t}{\sum_t \text{GT}_t}$$

where FN_t is the number of false negatives, FP_t the number of false positives, IDS_t that of identity switches, and GT_t is the number of ground truth individuals in frame t .

Multi-Object Tracking Precision (MOTP) MOTP measures the alignment of tracked objects by calculating the average distance error for correctly matched targets. Note that in contrast to MPJPE this metric is only computed over the true positives matched within the 15 centimeters maximum threshold.

$$\text{MOTP} = \frac{\sum_{t,d} \frac{1}{17} \sum_{k=1}^{17} \|\mathbf{p}_{t,d,k}^{\text{pred}} - \mathbf{p}_{t,d,k}^{\text{gt}}\|_2}{\sum_t \text{TP}_t}$$

where TP_t is the number of true positives.

Precision and Recall Finally, we also report the standard precision and recall metrics.

$$\text{Precision} = \frac{\sum_t \text{TP}_t}{\sum_t \text{TP}_t + \text{FP}_t}, \quad \text{Recall} = \frac{\sum_t \text{TP}_t}{\sum_t \text{TP}_t + \text{FN}_t}$$

5 Results and Discussion

This section of the report will present the evaluation results obtained by the developed system on the chosen test sequences. It further contains also discussion of the presented

results and possible reasons for the observed deficiencies.

5.1 Runtime Efficiency

Table 1 shows the mean per-frame runtime of the developed tracking system across the selected test sequences for different camera count and model capacity configurations. It can be seen that across every single configuration, the detection step consumes the vast majority of the processing time. For the base setup with YOLO26n-pose and 4 camera views, detection takes 0.0411 seconds out of a 0.0616-second total which equals approximately 67% of the frame time. This discrepancy becomes wider when increasing the model capacity. For the largest model, YOLO26l-pose, still with only 4 cameras, the detection takes up to 0.3408 seconds out of a total of 0.3648 seconds, which represents approximately 93% of the time.

When adding more camera views to the system using the smallest model, the computational load predictably increases. Because the detection model has to run a forward pass on every single camera feed, detection time scales almost linearly with the number of cameras, from 0.041s for 4 cameras to 0.192s for 16. Steps like matching and EKF updates also scale up as camera counts increase because these need to handle more cross-camera data points to associate and filter. However, their growth is somewhat slower than the linear increase for detection. The EKF prediction step is largely a negligible part of the overall system runtime.

When keeping the camera count at 4 but upgrading the YOLO architecture to larger capacity version, it can be observed that the cost increases significantly. Upgrading from YOLO26n-pose to YOLO26s-pose doubles the detection time, going from 0.0411s to 0.1137s. However, jumping to

Scene	MOTA	MOTP	MPJPE	Precision	Recall
160224_haggling1	0.861	3.974	4.961	0.900	0.971
160906_ian1	0.607	4.758	9.149	0.779	0.839
160906_pizza1	0.823	5.595	8.112	0.909	0.915
161029_build1	0.950	4.070	4.363	0.976	0.974
161029_sports1	0.631	4.291	9.278	0.783	0.869
170221_haggling_m3	0.943	3.869	4.198	0.960	0.984
170915_office1	0.994	4.114	4.136	0.997	0.998
171026_pose3	0.993	2.993	3.035	0.995	0.997

Table 3: Evaluation results for different scenes averaged over all performed evaluations of the baseline system. Evaluation metrics have been calculated as described in Table 2.

Configuration	MOTA	MOTP	MPJPE	Precision	Recall
Baseline	0.781	5.488	8.250	0.890	0.890
Constraints	0.842	5.139	7.264	0.920	0.923
Learned Kalman	0.880	4.679	5.801	0.937	0.944
Custom YOLO	0.886	3.788	5.318	0.941	0.946
LK + CY	0.893	4.083	5.184	0.944	0.950
Tuned	0.897	3.840	4.917	0.948	0.949

Table 4: Evaluation results for configurations of the system. All values are given for a setup with four cameras and the YOLO26n-pose model. Evaluation metrics have been calculated as described in Table 2. The best result in each column is highlighted in bold.

the medium or large version causes a massive spike, tripling the time to over a third of a second. Predictably, only the runtime for the detection step is significantly affected by increasing the model capacity and matching, EKF update, and Prediction remain almost entirely unchanged across all four model variants.

With the fastest configuration tested, the system achieves only around 16.2 fps. This is likely insufficient for real-time applications. However, it should be noted that not much effort has gone into optimization of the implementation, and that the tests were performed on relatively weak integrated GPU hardware.

5.2 Quantitative Results

Quantitative results for the baseline system on the chosen test sequences can be found in Table 2. When keeping the model fixed to YOLO26n-pose and increasing the number of cameras from 4 to 16, a significant reduction in pose and localization errors can be observed. The joint position error decreases significantly from 8.250 cm down to 2.834 cm. This indicates that incorporating more viewpoints substantially improves 3D localization and pose estimation accuracy. Furthermore, the increase in camera views brings improvements in recall. Recall increases from 0.890 with 4 cameras to 0.996 when using 16 cameras, indicating fewer missed detections likely because the cameras now cover more angles that would otherwise not be visible. Similarly, also precision increases initially when adding more camera views. However, adding even more cameras leads to a decrease in precision from 0.939 for 8 cameras to 0.934 with 16. This can be explained by the fact that more camera views introduce a higher likelihood of false positives and could also indicate limitations in the association algorithm. MOTA hits a sweet

spot of 0.926 with 12 cameras for the chosen test sequences.

In contrast, when keeping the camera count fixed at 4 and scaling up the model complexity from YOLO26n-pose to YOLO26l-pose, we see much smaller improvements when compared to adding additional cameras. The largest model achieves the best overall MOTA of 0.809 and precision of 0.915. Upgrading the model capacity also reduces MPJPE from 8.250 cm to 7.674 cm.

Next, Table 3 shows the average performance per scene across performed evaluations. It can be seen that the 160906_ian1 and 161029_sports1 scenes are the ones with the lowest performance metrics. In these scenes, MOTA and Precision suffer from a large increase in both false positives and false negatives. For 160906_ian1 the reason for the poor performance is explained by the fact that it contains a small child, which is often not well visible in the cameras, especially when only four cameras are in use. 161029_sports1 is a scene where people are performing sports related actions which may be difficult because of quick movements. The system performs particularly well on 171026_pose3 and 170915_office1. This is largely because in these scenes the persons are alone in the relative center of the room for long stretches of the video, avoiding a big source of errors for the system.

Finally, Table 4 shows how different more advanced configurations of the system perform compared to the baseline evaluated above. The MOTA score increases continuously from 0.781 at the baseline to a peak of 0.897 for the fully tuned configuration, representing a 0.116 gain. The largest single jump in MOTA performance comes from the introduction of the skeletal constraints. MPJPE drops from 8.250 cm down to 4.917 cm in the tuned setup, constituting a 40.4% overall error reduction. Here the largest improvement comes from the custom YOLO head. Both learned Kalman and custom YOLO individually provide significant perfor-

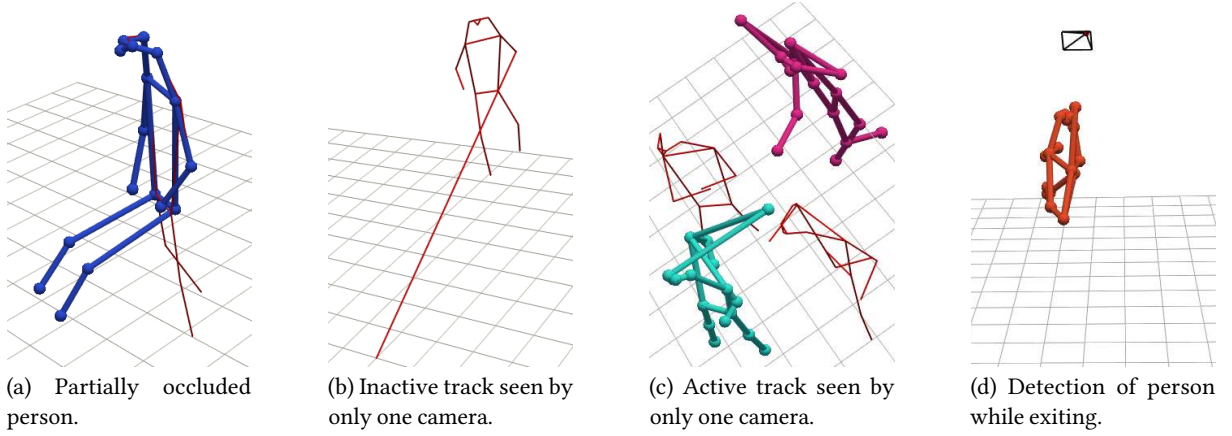


Figure 3: Some qualitative samples extracted from the evaluation results on the test set that exhibit commonly seen issues with the tracking system. Ground truth skeletons are drawn using thin red lines, while predicted skeletons are drawn thicker and with different colors per track.

mance gains, with the custom YOLO head providing slightly more performance when used individually. Combining both improves the performance further beyond their individual performance. Tuning the system thresholds yields the top-performing configuration overall, capturing a MOTA score of 0.897 and MPJPE of 4.917 cm.

5.3 Qualitative Results

Some qualitative evaluation has also been performed for this project.

5.3.1 Common Failure Modes

The results of the system on the tests sequences has been visualized and a number of common issues have been identified. Some examples of these failure modes are shown in Fig. 3.

Partially Occluded Person One very common failure mode is depicted in Fig. 3a. When a person is partially occluded in all camera views, then the position of the unobserved keypoints tends to shift in unnatural ways, as can be seen with the legs keypoints in the given example. This problem is made worse by the fact that the YOLO model tend to predict invisible keypoints in unnatural positions, and while their visibility score is low, it can still cumulatively have a significant effect over multiple frames. The issue causes increased joint position errors. This issue is somewhat resolved by either the learned Kalman filter and the custom YOLO head. The learned Kalman filter is able to resolve it by better modeling the behavior of unobserved states, while the custom YOLO head compensates by also giving better predictions for keypoints that are obscured or outside the image boundaries.

Inactive Track Seen by Only One Camera Another commonly observed failure is shown in Fig. 3b. When a person first enters the room, typically only one camera can see them. This means that it is not possible to create a new track for that person. This causes false positives, and therefore reduced MOTA and Recall, whenever a person enters the room. None of the proposed configurations are able to compensate for this issue.

Active Track Seen by Only One Camera Figure 3c shows another issue that can occur when only a single camera is able to see a given person. In this case, the track has already been initialized, but the person has moved into a location where they are no longer seen by two cameras. In this case the depth coordinate from the camera can drift as the person moves, causing distorted skeletons before and behind the actual ground truth. This problem is mitigated by the introduction of skeletal constraints, because it forces the skeleton to retain approximately the same size.

Detection of Person While Exiting Figure 3d shows a failure that occurs when a person leaves the room. In this case, the camera that is pointing at the exit will see the person for longer than the actual ground truth indicates. This causes false positives and therefore worse MOTA and Precision. Note that in this case we also have the above-mentioned issue of an active track seen by only one camera.

5.3.2 Impact of the Skeletal Constraints

The addition of skeletal constraints can help significantly improve the tracking performance especially in difficult situations where people are only partially visible. The choice of making the constraints be symmetric also helps in cases where only one side of a person is fully visible. Figure 4 shows a scene from the SALSA dataset. In Fig. 4a it shows the result obtained when using the baseline configuration of the system, while Fig. 4b shows the exact same scene but with the tracking system configured to use skeletal constraints. It is clearly evident that the skeletal constraints help prevent individual keypoints of partially occluded people from being pulled away from the rest of the skeleton, because in order to do that the limb length would have to increase.

6 Conclusion and Outlook

In conclusion, in this project a complete end-to-end 3D multi-person tracking pipeline was designed, implemented, and evaluated. The quantitative evaluation on the CMU Panoptic dataset provided some architectural insight. Spatial viewpoint density exhibits a far higher return on

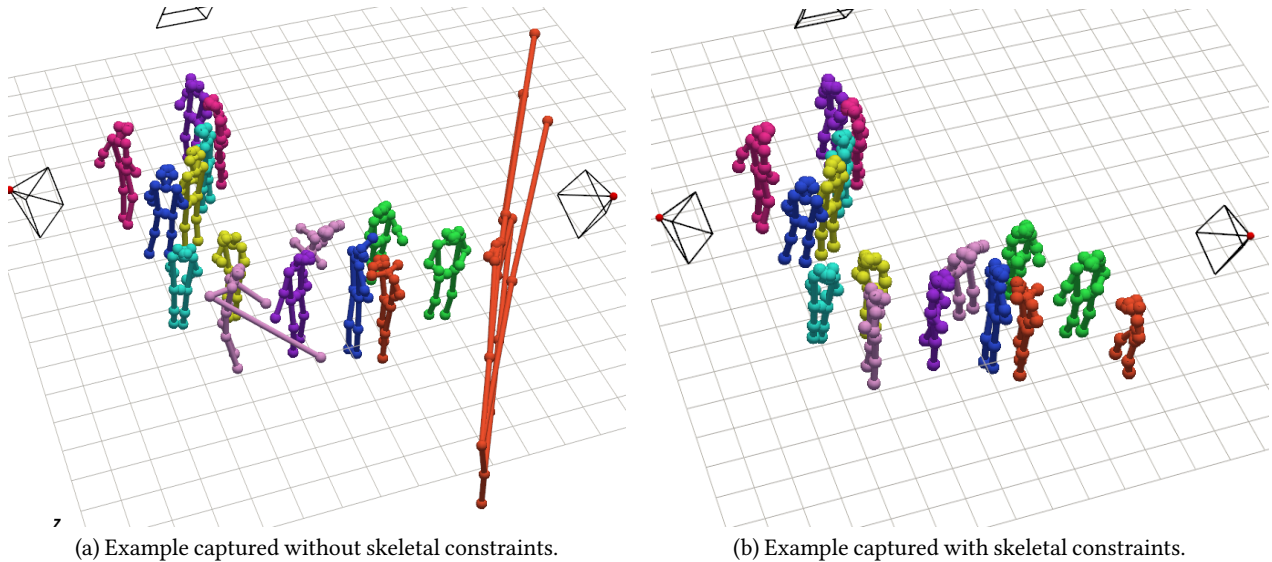


Figure 4: Two example scenes showing the impact of using skeletal constraints for the results obtained on the exact same segment of the SALSA dataset.

tracking accuracy than increasing deep learning model capacity. Scaling the camera view count from 4 to 16 with the YOLO26n-pose model reduced the mean per-joint position error from 8.25 cm down to 2.83 cm. In contrast, replacing this lightweight model with the heavy YOLO26l-pose model under a fixed 4-camera setup yielded smaller accuracy gains at the cost of a massive computational bottleneck, dropping system throughput to approximately 3 FPS. Additionally, performance can also be significantly improved with the addition of skeletal constraints as well as the use of learning from data for different components.

Despite the relatively high localization accuracy of the system under optimal conditions, there remain several limitations. As demonstrated in the qualitative evaluations, tracking quality degrades when targets are partially occluded or not in view of the camera views. Since 3D initialization requires triangulation across at least two views, the system also exhibits tracking delays during entry and spatial drifting during single-camera visibility. Further, the way the system has been adapted to the dataset by training individual components separately limits the amount of adaptation.

Possible future improvements to address these limitations could include fully end-to-end training of the system parameters from the dataset instead of training components separately. A further improvement could be the addition of further anatomical plausibility constraints on the state predictions, such as realistic joint angles. These constraints could possibly be included similar to the existing limb length constraints using a pseudo-measurement. Furthermore, the addition of additional information sources, such as monocular depth values could make it possible to track people even using single camera views. Exploring more advanced filtering methods such as particle filters, potentially also using occlusion aware likelihoods could be promising future directions. Finally, the evaluation in this project has been performed only on the CMU Panoptic dataset. As such, the evaluation of the system using a larger set of more diverse sequences would be highly valuable to uncover further areas for improvement.

References

- Alameda-Pineda, Xavier, Jacopo Staiano, Ramanathan Subramanian, Ligia Batrinca, Elisa Ricci, Bruno Lepri, Oswald Lanz, and Nicu Sebe (2015). “Salsa: A novel dataset for multimodal group behavior analysis”. In: *IEEE transactions on pattern analysis and machine intelligence* 38.8, pp. 1707–1720.
- Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. (2024). “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation”. In: *Proceedings of the 29th ACM international conference on architectural support for programming languages and operating systems, volume 2*, pp. 929–947.
- Bar-Shalom, Yaakov, X Rong Li, and Thiagalingam Kirubarajan (2001). *Estimation with applications to tracking and navigation: theory algorithms and software*. John Wiley & Sons.
- Bernardin, Keni and Rainer Stiefelhagen (2008). “Evaluating multiple object tracking performance: the clear mot metrics”. In: *EURASIP Journal on Image and Video Processing* 2008.1, p. 246309.
- Bewley, Alex, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft (2016). “Simple online and realtime tracking”. In: *2016 IEEE international conference on image processing (ICIP)*. Ieee, pp. 3464–3468.
- Bharati, Puja and Ankita Pramanik (2019). “Deep learning techniques—R-CNN to mask R-CNN: a survey”. In: *Computational Intelligence in Pattern Recognition: Proceedings of CIPR 2019*, pp. 657–668.
- Dondrup, Christian, Nicola Bellotto, Ferdian Jovan, Marc Hanheide, et al. (2015). “Real-time multisensor people tracking for human-robot spatial interaction”. In: *Workshop on machine learning for social robotics at international conference on robotics and automation (ICRA)*, p. 209.
- Gaya-Morey, F Xavier, Cristina Manresa-Yee, and José M Buades-Rubio (2024). “Deep learning for computer vision

- based activity recognition and fall detection of the elderly: a systematic review: GM F. Xavier et al." In: *Applied Intelligence* 54.19, pp. 8982–9007.
- Girshick, Ross (2015). "Fast r-cnn". In: *Proceedings of the IEEE international conference on computer vision*, pp. 1440–1448.
- Han, Xiaofeng, Diego Guffanti, and Alberto Brunete (2025). "A comprehensive review of vision-based sensor systems for human gait analysis". In: *Sensors* 25.2, p. 498.
- Hartley, Richard and Andrew Zisserman (2003). *Multiple view geometry in computer vision*. Cambridge university press.
- He, Kaiming, Georgia Gkioxari, Piotr Dollár, and Ross Girshick (2017). "Mask r-cnn". In: *Proceedings of the IEEE international conference on computer vision*, pp. 2961–2969.
- Ionescu, Catalin, Dragos Papava, Vlad Olaru, and Cristian Sminchisescu (2013). "Human3.6m: Large scale datasets and predictive methods for 3d human sensing in natural environments". In: *IEEE transactions on pattern analysis and machine intelligence* 36.7, pp. 1325–1339.
- Jiang, Peiyuan, Daji Ergu, Fangyao Liu, Ying Cai, and Bo Ma (2022). "A Review of Yolo algorithm developments". In: *Procedia computer science* 199, pp. 1066–1073.
- Jocher G. Chaurasia A., Qiu J. (2026). *Ultralytics YOLO*. <https://github.com/ultralytics/ultralytics>. GitHub Repository.
- Joo, Hanbyul, Hao Liu, Lei Tan, Lin Gui, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh (2015). "Panoptic Studio: A Massively Multiview System for Social Motion Capture". In: *The IEEE International Conference on Computer Vision (ICCV)*.
- Joo, Hanbyul, Tomas Simon, Xulong Li, Hao Liu, Lei Tan, Lin Gui, Sean Banerjee, Timothy Scott Godisart, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh (2017). "Panoptic Studio: A Massively Multiview System for Social Interaction Capture". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Koppula, Hema S and Ashutosh Saxena (2015). "Anticipating human activities using object affordances for reactive robotic response". In: *IEEE transactions on pattern analysis and machine intelligence* 38.1, pp. 14–29.
- Kuhn, Harold W (1955). "The Hungarian method for the assignment problem". In: *Naval research logistics quarterly* 2.1-2, pp. 83–97.
- Kurillo, Gregorij, Evan Hemingway, Mu-Lin Cheng, and Louis Cheng (2022). "Evaluating the accuracy of the azure kinect and kinect v2". In: *Sensors* 22.7, p. 2469.
- Lanz, Oswald (Spring 2026). *Computer Vision: Lecture Notes*. Course lecture notes for Free University of Bolzano course number 73075. Accessed June 10, 2026.
- Lin, Tsung-Yi, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick (2014). "Microsoft coco: Common objects in context". In: *European conference on computer vision*. Springer, pp. 740–755.
- Luo, Wenhan, Junliang Xing, Anton Milan, Xiaoqin Zhang, Wei Liu, and Tae-Kyun Kim (2021). "Multiple object tracking: A literature review". In: *Artificial intelligence* 293, p. 103448.
- Maji, Debapriya, Soyeab Nagori, Manu Mathew, and Deepak Poddar (2022). "Yolo-pose: Enhancing yolo for multi person pose estimation using object keypoint similarity loss". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 2637–2646.
- Mehta, Dushyant, Oleksandr Sotnychenko, Franziska Mueller, Weipeng Xu, Mohamed Elgharib, Pascal Fua, Hans-Peter Seidel, Helge Rhodin, Gerard Pons-Moll, and Christian Theobalt (2020). "XNect: Real-time multi-person 3D motion capture with a single RGB camera". In: *Acm Transactions On Graphics (TOG)* 39.4, pp. 82–1.
- Nogueira, Ana Filipa Rodrigues, Hélder P Oliveira, and Luis F Teixeira (2025). "Markerless multi-view 3D human pose estimation: A survey". In: *Image and Vision Computing* 155, p. 105437.
- Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. (2019). "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32.
- Ren, Shaoqing, Kaiming He, Ross Girshick, and Jian Sun (2015). "Faster r-cnn: Towards real-time object detection with region proposal networks". In: *Advances in neural information processing systems* 28.
- Sapkota, Ranjan, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee (2025). "YOLO26: key architectural enhancements and performance benchmarking for real-time object detection". In: *arXiv preprint arXiv:2509.25164*.
- Sapkota, Ranjan and Manoj Karkee (2025). "Ultralytics YOLO evolution: An overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 object detectors for computer vision and pattern recognition". In: *arXiv preprint arXiv:2510.09653*.
- Simon, Tomas, Hanbyul Joo, and Yaser Sheikh (2017). "Hand Keypoint Detection in Single Images using Multiview Bootstrapping". In: *CVPR*.
- Sultani, Waqas, Chen Chen, and Mubarak Shah (2018). "Real-world anomaly detection in surveillance videos". In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, pp. 6479–6488.
- Svenstrup, Mikael, Soren Tranberg, Hans Jorgen Andersen, and Thomas Bak (2009). "Pose estimation and adaptive robot behaviour for human-robot interaction". In: *2009 IEEE International Conference on Robotics and Automation*. IEEE, pp. 3571–3576.
- Virtanen, Pauli, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature methods* 17.3, pp. 261–272.
- Zheng, Ce, Wenhan Wu, Chen Chen, Taojiannan Yang, Sijie Zhu, Ju Shen, Nasser Kehtarnavaz, and Mubarak Shah (2023). "Deep learning-based human pose estimation: A survey". In: *ACM computing surveys* 56.1, pp. 1–37.